Graph intelligence

When the relationships are the data

M Usman

An applied view of learning from connected data.

Where this is heading

Some problems are not rows. They are the relationships between them.

M Usman · From enterprise data to production AI.

The signal is in the connections

- Gartner predicted that graph technologies would be used in **80% of data and analytics innovations by 2025**, up from **10% in 2021** (Gartner, 2021).
- The problem they address is large: an estimated **$3.1 trillion** in illicit funds moved through the financial system in 2023 (Nasdaq, 2024), much of it visible only as a pattern across accounts, not in any single record.
- Graph-grounded retrieval now lifts LLM answers on whole-corpus questions (Microsoft Research, 2024), and the graph-database market is growing at roughly **27% a year** (Mordor Intelligence, 2025).

> When the signal lives in how things connect, a graph is the substrate, not a storage choice.

The slow step

Row-based methods cannot see a shape that spans many records

A fraud ring, an ownership chain, a lateral-movement path: the evidence is a multi-hop pattern no single row holds. A graph makes that pattern queryable. But the value rests entirely on one upstream decision, resolving which records are the same real-world entity, and that is where most efforts quietly break.

> The slow step is the topology itself: a graph is only as good as the entity resolution beneath it.

The stack at a glance

1 · Model
the domain as a graph
leaks as: a graph where a table fits

›

2 · Resolve
entities, deduplicated
leaks as: resolution errors poison all

›

3 · Learn
algorithms · GNNs
leaks as: leakage from target edges

›

4 · Serve
score · evidence subgraph
leaks as: a score nobody can explain

Each stage has one job, one common way to lose the value, and one discipline that protects it. The next four slides take them in turn.

M Usman · From enterprise data to production AI.

1 · Model the domain as a graph

Decide what the things and connections are, and encode them so the relationships, not the rows, carry the signal. This stage fixes which questions are cheap and which become impossible.

Practical steps

- Start from the traversal questions the business asks, not from the existing tables
- Apply the node/edge/property test; promote a relationship that carries signal to its own node
- Choose property graph (fast traversal, edge attributes) or RDF (standards, reasoning) deliberately
- Write the identity contract first: what makes two records the same node

The pitfall: a schema built around today's questions. A model tuned for one set of traversals makes the next set O(n) or impossible, and migration after load is costly. The sibling error is forcing a graph where a fixed-depth lookup belongs in a table.

What good looks like. Query-led, and reified where it matters: things are nodes, the signal-carrying connection is a first-class indexable object, the identity contract is explicit. A graph earns its keep only when traversal depth is variable.

Illustrative tools: Neo4j, TigerGraph, Memgraph (property graph) · Apache Jena, GraphDB (RDF) · arrows.app, OWL/SHACL.

2 · Resolve entities and build

Turn many noisy records that refer to real-world entities into one node per actual entity, then ingest into a graph store that holds at scale.

Practical steps

- Block to make it tractable: partition records so only plausible pairs are compared, tuned for recall
- Score candidate pairs (Fellegi–Sunter likelihood, or learned matchers) per attribute, then combine
- Cluster pairwise matches into consistent entities; reconcile A=B and B=C but A≠C
- Set thresholds against the asymmetric cost of a false merge versus a false split; keep a review band

The pitfall: a merge or split that poisons everything. A false merge invents relationships; a false split hides coordinated behaviour. Every later stage inherits it, and the damage only surfaces as an unexplainable wrong answer three stages on.

What good looks like. Treat entity resolution as the load-bearing wall: measure it with cluster-level metrics on a gold set, keep humans in the ambiguous band, and hold full reversible lineage.

Illustrative tools: Splink, Dedupe, Zingg, Senzing (resolution) · Neo4j, TigerGraph, Nebula (store) · Spark, Kafka (ingest).

3 · Learn on the graph

Extract signal from the structure itself: first deterministic algorithms (who is central, what clusters exist), then graph neural networks that learn by passing messages between neighbours.

Practical steps

- Start with classical algorithms: PageRank and betweenness for influence and brokers, Leiden for communities
- Engineer cheap graph features (degree, triangles, hop-to-known-bad) before any GNN
- Pick the architecture to the job: attention when neighbours differ in importance, sampling aggregators for scale
- Choose inductive over transductive if you will score new entities, and manage depth against over-smoothing

The pitfall: leakage in link prediction. If the edges you predict stay in the message-passing graph at train or test, aggregation leaks the answer and the score is fiction, worst for the low-degree nodes you care about most.

What good looks like. Split the graph, not the rows: remove target edges from the structure, use a temporal split for evolving graphs, hard negatives, and a metric matched to the deployment. Always benchmark the GNN against the simple, explainable baseline.

Illustrative tools: Neo4j GDS, NetworkX, cuGraph (algorithms) · PyTorch Geometric, DGL (GNNs) · node2vec (embeddings).

4 · Serve and explain

Put the structure to work at decision time: pattern queries for investigation, real-time scoring of new nodes, an explanation a human can act on, and monitoring as the graph drifts.

Practical steps

- Expose two paths: declarative pattern queries for investigation, and model scoring behind an API
- Precompute the deep features; traverse shallow at request time, with a hard hop limit
- Score new entities inductively in place, pulling only their k-hop neighbourhood
- Attach the supporting subgraph to every score, and feed reviewed outcomes back as labels

The pitfall: a black-box score nobody can action. A risk score with no supporting subgraph is uninvestigable and, in regulated domains, non-compliant; deep multi-hop traversals that miss the latency budget are its operational twin.

What good looks like. Every score ships with its evidence subgraph in the investigator's language; latency is met by precomputing depth and bounding hops; the graph is monitored for drift, and reviewed outcomes refresh resolution and the model.

Illustrative tools: Cypher/GQL, SPARQL, Gremlin (queries) · PyG/DGL behind FastAPI (serving) · GNNExplainer (explanation).

The maturity ladder

5 · GRAPH LEARNING
GNNs, embeddings, link prediction, served inductively and explained, outcomes looping back

4 · GRAPH ANALYTICS
centrality, community detection, pathfinding over the graph

3 · GRAPH STORE
relationships first-class and indexed; variable-depth traversals are native and fast

2 · JOINS AT BREAKING POINT
recursive self-joins to walk relationships; performance falls off past two or three hops

1 · TABLES
relationships implicit, reconstructed by joins at query time

Most teams sit at rung 2 and mistake it for a tuning problem. The jump 2 to 3 is an architecture decision, 4 to 5 an ML and governance one; and rung 5 has a hard dependency on stable entity resolution two stages back.

Four pitfalls that span the whole stack

- **Entity-resolution error poisons everything.** A false merge or split corrupts the topology every later stage trusts; invisible where it happens, it surfaces as an inexplicable wrong answer at serving.
- **A graph where a table suffices.** Graph cost is justified only when traversal depth is variable and relationships are first-class; fixed lookups and aggregations belong in a relational or columnar store.
- **Leakage in graph evaluation.** Target edges left in the message-passing or test graph, or a changed snapshot, inflate offline metrics and then collapse in production.
- **An unexplainable score.** A prediction with no supporting subgraph cannot be triaged, appealed, or accepted in regulated work; latency on deep traversals is its operational twin.

The lens behind this view

My doctoral research is built on graph attention networks: a failure-aware architecture that reasons over the relationships between tasks and compute resources, not the records alone. My MSc work built knowledge graphs from text, with named-entity extraction feeding the graph, and explored biomedical knowledge graphs in a healthcare seminar.

The shift this deck describes, from rows to relationships, is the one my own work runs on.

> The research and the engineering behind it are public: mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.

In one line

When the signal lives in how things connect, the relationships are the data, not an afterthought. Model them as a graph, resolve the entities cleanly, learn on the structure without leaking it, and serve a score you can explain. That is the difference between storing connections and acting on them.

M Usman · From enterprise data to production AI.

Sources

Gartner, Top 10 Data & Analytics Trends (2021): graph technologies in 80% of data-and-analytics innovations by 2025, up from 10% in 2021 · Nasdaq / Verafin, Global Financial Crime Report (2024): ~$3.1tn estimated illicit funds in 2023 · Microsoft Research, GraphRAG (2024): graph-grounded retrieval gains on whole-corpus benchmarks · graph-database market ~27% CAGR (Mordor Intelligence, 2025) · Method grounded in the property-graph, entity-resolution (Fellegi–Sunter) and GNN literature (GraphSAGE, GAT; the link-prediction leakage benchmarks).

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman

See the full portfolio

The work behind this deck sits on one page: the Insights series, the projects and the code that runs them, and the publications.

mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.